AI Curriculum — 3rd Year

3rd Year AI Curriculum

Module 3.1 — Why Code When You Can Vibe Code? (The Bridge)

This is the transition module. Addresses the obvious question: if vibe coding works, why learn to code at all? Covers what breaks when vibe coding fails, why understanding the code underneath makes you 10x better at vibe coding, and what Python actually adds that plain English prompting can't.

Module 3.2 — Python Fundamentals for AI

Variables, data types, lists, dictionaries, loops, functions. Not full Python — only what's needed for AI work. Google Colab setup. Running your first script. Reading and understanding AI-generated Python code.

Module 3.3 — Python Libraries for AI

NumPy, Pandas, Matplotlib — the three essentials. Reading a CSV, cleaning data, basic aggregation, plotting a chart. Everything done in Colab so no local setup friction.

Module 3.4 — Working with AI APIs

What an API is, HTTP requests (GET/POST), JSON, API keys and why they must never go in a public repo. Hands-on with Gemini free tier or OpenAI API. Reading API documentation.

Module 3.5 — Build a Chatbot

Using what they learned in previous modules, students build a Python chatbot that takes user input and returns AI responses via API. First real coded project pushed to GitHub with a proper README.

Module 3.6 — Data Basics & Cleaning

What messy real-world data looks like. Reading CSVs with Pandas. Handling nulls, duplicates, wrong data types. Grouping, filtering, aggregating. Where to find public datasets such as Kaggle and data.gov.in.

Module 3.7 — Data Visualisation & Storytelling

Matplotlib and Seaborn charts: bar, line, scatter, heatmap. Choosing the right chart for the right data. Adding titles, labels, annotations. Making a chart that communicates something clearly.

Module 3.8 — Introduction to Machine Learning

What ML is under the hood — training, testing, features, labels, models. Scikit-learn basics. Linear regression and classification — conceptual understanding plus simple code. Using pre-trained models from Hugging Face. No math-heavy theory; purely applied.

Module 3.9 — Build an ML Project

End-to-end mini ML project. Student picks a public dataset, defines a problem such as predicting house prices or classifying spam, trains a simple model, evaluates it, and documents findings. Full repo pushed to GitHub with README explaining the problem, approach, and results.

Module 3.10 — Deploy Your AI Project

Taking a working Python project and making it accessible to others. Streamlit for building simple UIs around AI/ML code. Hugging Face Spaces for free hosting. Students deploy their Module 3.9 project live and update GitHub repo with the live link.

Module 3.11 — Advanced Python for AI

Object-oriented programming: classes, objects, inheritance, and why it matters for bigger projects. List comprehensions, decorators, error handling, file I/O. Writing clean, readable, documented code. Virtual environments and dependency management using pip and requirements.txt.

Module 3.12 — Deep Dive into Data

Beyond basic Pandas. Merging and joining datasets, pivot tables, time series data, handling large files efficiently. Feature engineering. Exploratory data

analysis as a structured process. Statistical basics — mean, median, distribution, correlation — explained through code.

Module 3.13 — Machine Learning: Going Deeper

Decision trees, random forests, gradient boosting/XGBoost, clustering/K-means, dimensionality reduction/PCA. Proper model evaluation: confusion matrix, precision, recall, F1 score, ROC curve. Overfitting and underfitting. Cross-validation and hyperparameter tuning basics.

Module 3.14 — Neural Networks & Deep Learning Fundamentals

Neurons, layers, weights, activation functions. Forward pass and backpropagation explained simply. Introduction to TensorFlow and Keras. Building a simple neural network from scratch. Training on a real dataset such as MNIST. When to use deep learning vs classical ML.

Module 3.15 — Natural Language Processing (NLP)

Text as data: tokenisation, stopwords, stemming, lemmatisation. Bag of words and TF-IDF. Sentiment analysis classifier. Introduction to transformers. Hugging Face pipelines for text classification, summarisation, translation, and question answering.

Module 3.16 — Working with LLMs Professionally

System prompts, temperature, top-p, max tokens. Prompt chaining. Structured outputs and JSON. Function calling basics. Introduction to RAG and simple implementation using a vector store. LangChain fundamentals: chains, agents, tools, memory.

Module 3.17 — Computer Vision Basics

Images as data: pixels, channels, resolution. OpenCV for reading, resizing, filtering, edge detection. Pre-trained vision models: YOLO for object detection and CLIP for image-text matching. Simple image classifier using transfer learning. Use cases: facial recognition, medical imaging, manufacturing quality control.

Module 3.18 — AI Automation & Agents

Beyond chatbots — AI that takes actions. Introduction to agents, tools, and workflows. Building a simple AI agent using LangChain or CrewAI that can

browse, search, summarise, and respond. No-code automation with n8n or Make.com for workflows involving email, calendar, Notion, or WhatsApp.

Module 3.19 — Building Full-Stack AI Applications

Python AI backend plus front end. FastAPI for AI APIs. React or simple HTML frontend connected to backend. User authentication basics. Database basics with SQLite or Supabase. Students build a complete web application with real UI, AI feature, and persistent data.

Module 3.20 — Advanced AI Project

Student builds a production-quality AI application solving a real problem for real users. Must include proper front end, AI component, clean GitHub repo, documentation, live deployment URL, and 5-minute demo video. This becomes a resume/GitHub portfolio project.

Module 3.21 — RAG (Retrieval Augmented Generation) — Deep Dive

Build RAG properly from scratch. Covers why RAG exists: hallucinations, knowledge cutoffs, and private data access. Full pipeline: document ingestion, chunking strategies, embedding models, vector databases, retrieval, MMR, re-ranking, context injection, and generation. Build a chatbot answering questions about a real document. Evaluate RAG quality using context relevance, answer faithfulness, and answer relevance. Common RAG failure modes and fixes.

Module 3.22 — Advanced LLM Engineering

LLM internals: transformers, attention, embeddings, tokens, context window. Advanced prompting: meta-prompting, self-consistency, tree of thought, ReAct. Structured outputs using Pydantic and LLMs. LLM routing, caching, multi-modal LLMs, and comparing frontier models such as GPT-4o, Gemini, Claude, and Llama.

Module 3.23 — AI Agents & Multi-Agent Systems

Planning, tool use, memory, self-correction. Agent architectures: ReAct, Plan-and-Execute, reflection-based agents. LangChain and LangGraph: nodes, edges, state management, conditional routing. CrewAI multi-agent systems. Tool use: search web, read files, write code, call APIs, send emails. Memory systems: short-term, long-term vector memory, entity memory. Build a research agent.

Module 3.24 — Fine-Tuning & Custom Models

When fine-tuning is necessary vs when RAG or prompting is sufficient. Full fine-tuning, LoRA, QLoRA. Dataset preparation, instruction tuning format, data quality, and quantity. Fine-tune a small open-source model such as Llama or Mistral using Hugging Face and free GPU options. Evaluate against base model. Deploy on Hugging Face Spaces. Compare OpenAI fine-tuning API vs open-source fine-tuning.

Module 3.25 — Vector Databases & Embeddings — Production Level

Embedding models: OpenAI embeddings, Sentence Transformers, BGE models, choosing the right model. Vector DB internals: HNSW indexing and approximate nearest neighbour search. Production vector DB setup with namespaces, metadata filtering, hybrid search. Build a semantic search engine. Multi-modal embeddings. Embedding evaluation.

Module 3.26 — MLOps & Production AI Systems

The gap between a project and production reliability. Experiment tracking with MLflow or Weights & Biases. Model versioning and registry. CI/CD for ML using GitHub Actions. Monitoring: data drift, model drift, performance degradation. Retraining safely. Docker basics for AI apps. Cloud deployment using AWS SageMaker or Google Cloud Vertex AI. Cost optimisation.

Module 3.27 — AI System Design

System design fundamentals for AI: latency, throughput, scalability, reliability, cost. Designing RAG for 100,000 users. Caching strategies: semantic caching, response caching, embedding caching. Load balancing across LLM providers. Fallback strategies. Async processing and queue-based architectures using Celery/Redis. Case studies: bank customer support AI, ed-tech AI tutor, law firm document analysis.

Module 3.28 — AI Security & Safety

Prompt injection, jailbreaking, model manipulation, data poisoning, PII leakage, content moderation, AI red-teaming, guardrails, and compliance including DPDP Act India and GDPR.

Module 3.29 — Capstone Preparation & Portfolio Polish

GitHub profile audit: pinned repos, README, commit history, documentation. Add demo GIFs, screenshots, deployment links. Write project case studies. Build personal AI portfolio website. LinkedIn optimisation. Prepare to explain projects and technical decisions.

Module 3.30 — 3rd Year Final Capstone: Production-Grade AI System

Final project must demonstrate at least 3 of: RAG pipeline, AI agents, fine-tuned model, full-stack deployment, MLOps practices, multi-modal AI, real users or real data.

Deliverables: full GitHub repo with clean documented code, live deployed application, technical write-up explaining architecture decisions/challenges/results, 7-minute demo video, and honest reflection.

Examples: AI-powered legal document analyser for Indian law, personalised study assistant using RAG over personal notes, multi-agent market research system, AI customer support system for a small business, or fine-tuned model for a regional Indian language.